J-Vision

Ben Meyers

July 2026

1 Through the Jacobian Lens

Anthropic recently published a paper introducing what they call the **J-space** ($\mathscr{S}$): a privileged set of directions in a Large Language Model's latent activation space, derived using the **J-lens** ($\mathscr{L}$). The **J** in both names stands for the **Jacobian**, a generalization of the derivative for multivariate functions on tensors. Given that deep neural networks are comprised almost exclusively of tensors and tensor transformations, Jacobians are nothing new. Interpretable *ends* (input and output) and *end-to-end* differentiability have been key prerequisites in the deep learning contract, because making desired changes in output- or loss-space requires knowing each weight tensor's causal impact *on* that output or loss, and differentiation is the analytical language of causal relationships.

Though whereas they're typically thought of primarily to guide training, this new work shows that end-to-end Jacobians can let us observe and interpret the networks we've wrought. Once we've reached a satisfactorily low level of loss and we've stopped adjusting *weights*, we can use the same mathematical tools to identify the causal weight of the *activations* in the trained model.[1] Further, since the *effect* we're interested in is the by-construction-interpretable realized behavior of the model, we can establish a direct, interpretable link between internal activity in the model and the domain in which we've chosen to situate it. We can pull that final layer—where the model chooses *words*, which stand for *concepts*—back to any hidden layer of the network—which, on its own, is an unintelligible high-rank tensor—and get a sense of what that hidden layer *portends* conceptually. The **J-lens** gives us a direct view into *disposition* of the model, as represented by its internal activations, as it performs its computation.

It is important to impress that the **J-lens** doesn't just *correlate* internal activations with external expressions. It does not offer a merely statistical picture of interpretability, where statements can be made to the effect of: "this circuit of neurons always seems to fire when the model discusses dogs, so therefore

[1] For the sake of completeness, it should be clarified that (*a*) interrogating *activations* as opposed to weights with Jacobians does not "unlock" only after training and (*b*) in order to take derivatives of upstream weights (for training), you already *must* propagate them through activations. The point is that during training we're more interested in derivatives with respect to weights and that after training, when weights freeze, it becomes more interesting to interrogate activations.

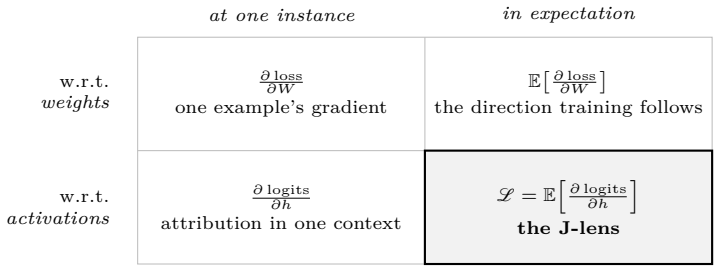

Figure 1: Four derivative objects from the same contract. The top row has always steered *training*; the bottom-left explains single predictions after the fact. The fourth square—activation Jacobians taken *in expectation*—is the new instrument.

this circuit may encode something about dogs." *Correlation*, we know platitudinally, does not imply *causation*. These networks are entirely deterministic; these Jacobians[2] speak only the language of analytically-derived *cause*. They allow us to say: "this bundle of activated neurons reflects that the model is disposed to mention dogs; if we amplify these currently-activated neurons, the model *will mention dogs*." Further, once the directions in the model's activation space are inflected conceptually, we can do better than just identify disposition implied by state. We can tap the tools from the abstract formalism of linear algebra to work—steer, perturb and observe, illustratively decompose—in a now-interpretable space more economically. *Calculus* colors abstract directions with purpose, *linear algebra* teaches us how to traverse, manipulate, and qualify them.

This paper will explain the techniques employed to define the **J-lens** and the **J-space** and will seek to elucidate how fruitful our chosen formulation for deep learning—differentiable vector spaces—is and can continue to be.

2 Transformer Architecture: Positions, Layers, Logits, and the Residual Stream

To appreciate the **J-lens** ($\mathscr{L}$) and the **J-space** ($\mathscr{S}$) it finds, a review of some principal components of the **Transformer** neural network architecture (that which carries out most modern LLMs) is in order. The key elements we'll discuss are:

1. The manner in which token **Position** is represented and handled in the transformer's computation
2. The way computation propagates forward through the **Layers** of the network
3. The key output of the network which represents its prediction for best choice of *next* token, the non-normalized vector of **Logits**

[2]Although they're, importantly, *averaged linear approximations*—much more on this later.

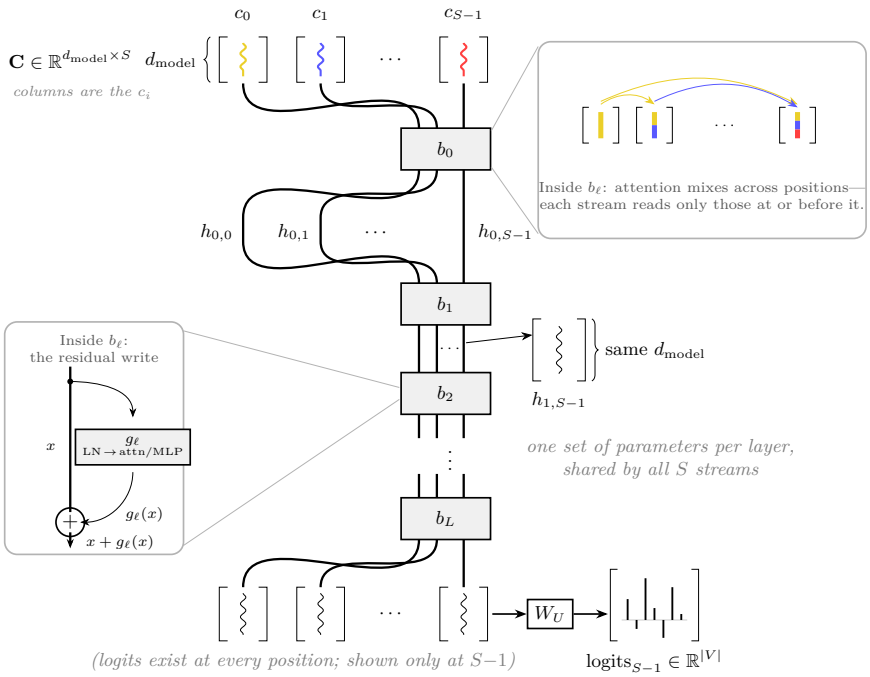

Figure 2: The transformer as one stack of blocks b_ℓ and S residual streams. The input is a matrix $\mathbf{C} \in \mathbb{R}^{d_{\text{model}} \times S}$ whose columns c_i are the token vectors; S is a free axis. Each layer is a single set of parameters; every position's stream passes through the same blocks, which is why the architecture is indifferent to sequence length. After b_0 the streams are pulled apart and named to show that each carries one position's state, $h_{\ell,i}$; thereafter they are drawn bundled for concision, preserving shape d_{model} throughout. The zoom shows where cross-position mixing happens: attention inside each block, under the causal mask. Colors trace content: each token enters in its own color, and after attention each stream holds a blend of every position at or before it—the final stream, a neapolitan of the whole context. The left zoom opens the block itself: the stream splits, the identity path carries x around, the learned transformation writes $g_\ell(x)$, and they rejoin at $\oplus$—every block computes $x + g_\ell(x)$ (§2.4). Only the final stream is unembedded here, though every stream admits it.

4. The cord which carries *state* through the layers of the model, the **Residual Stream**

At a high level, the transformer takes as input a context matrix, $\mathbf{C}$, where each column, c_i, represents a token of input as a vector. The transformer admits any sequence length, S, meaning $\mathbf{C}$'s column count is *free*. The dimension of each c_i, though, is fixed model-wide at d_{model}. The network's weights contract against d_{model}, so S naturally carries forward seamlessly through each layer.

Strictly speaking, the transformer outputs S final activation vectors, $h_{L,i}$, for all $i < S$.[3] For practical expository purposes, we're taking the model as trained and ready for autoregressive *generation*, so we really only care about $h_{L,S-1}$—the final position's final activation. "Care about" means decode via an unembedding matrix, $W_U \in \mathbb{R}^{|V| \times d_{\text{model}}}$, where V is the token vocabulary and $|V|$ its size. This produces a vector, $\text{logits}_{S-1} \in \mathbb{R}^{|V|}$, representing the weight assigned to every possible token in the vocabulary as a candidate to come *next* in the sequence.

The model, in other words, takes an entire context of S tokens—$\mathbf{C}$—and produces a single prediction of which token should come next. Strictly speaking, logits_{S-1} itself is not a probability distribution—it still awaits normalization via the *Softmax* function—but it is its direct proxy. Once we turn logits_{S-1} into a probability distribution via *Softmax*, we can sample the distribution, choose a new token, and create a new context matrix, $\mathbf{C} \in \mathbb{R}^{d_{\text{model}} \times (S+1)}$, where c_S is the vector representation of the new token.

2.1 Positions

In autoregressive generation only the final position's logits are decoded into the next token, though tokens at all positions must be fed into the model for any forward pass. Correspondingly, logits are produced at *every* position—in fact, during training, every position's logits receive a loss. Why include every prior position? The transformer architecture is such that each latent $h_{\ell,i}$ moving through the network is informed by all of the tokens at positions $j \leq i$. In each transformer block, b_ℓ, there is a module known as an **Attention Mechanism**, which facilitates the flow of information from the previous boundary's latents $h_{\ell-1,j}$ into $h_{\ell,i}$, for all positions $j \leq i$. Figure 2 articulates this information flow at a high level.

This forward-only flow of contextual information along the token axis is a vital element of the construction of the **J-lens**. Though logits_{S-1} is produced by decoding only $h_{L,S-1}$ directly, attention makes it such that a latent $h_{\ell,i}$ at *any* layer and position has a causal impact on logits_{S-1}.[4] The goal for the **J-lens** is to produce an average general map from any latent to all downstream logits; this

[3] A note on subscripts, used throughout: the first indexes the *layer boundary*—$h_{\ell,i}$ is the residual state at position i just after block b_ℓ, with L reserved for the final layer—and the second indexes the *position*.

[4] With one notable exception: final-layer latents at non-final positions, $h_{L,i}$ with $i < S-1$, are causal dead ends for logits_{S-1}, since no attention remains downstream of them to carry their contents across streams.

is precisely what enables it to inform *general verbalizable disposition*, as opposed to a more constrained quantity only pertaining to the logits at any latent's own position. §3.1 covers the precise mechanism in more detail, but in computing the **J-lens**, we take Jacobians and average them over *both* directions of the position axis, pairing every source latent $h_{\ell,t}$ with every logit vector it feeds, $\text{logits}_{t'}$ for $t' \geq t$ (this averaging, still, just within each prompt context, over which we *also* average).[5] The payoff is that each layer's lens $\mathscr{L}_\ell$, once computed, is not bound by position at all—we get one W_U-shaped map, applicable to any latent in the stream at layer ℓ.

2.2 Layers

Whereas attention provisions for information to move forward along the *token position* axis, a transformer's depth in *layers* allows for complex, highly nonlinear processing to occur on latents. Feedforward neural networks, in general, are compositions of functions, where each layer represents another function on the output of the layers before it. When layers are given nonlinear activation functions, their composition becomes highly nonlinear.

At each layer of a transformer, tokens are being processed at different levels of abstraction. For instance, early layers of transformers may be focused more on composing a *syntactic* representation of the context, which, in later layers, is then processed for its *semantic* and *factual* content, syntax having been a prerequisite. For this reason, having a **J-lens** at every layer provides distinct insight into leverage over logits_{S-1} at different levels of abstraction.

Further, because of the compositional nature of the transformer, the layers downstream from any specific layer, ℓ, can be considered a single nested function, F_ℓ, that maps $h_{\ell,i}$ to logits_{S-1} (with the rest of the context held fixed). This framing, displayed in Figure 3, becomes insightful for the purposes of computing and conceptualizing **J-lens**es.

2.3 Logits

As Figure 2 and the above discussions elucidate, the transformer is trained to produce final latents, $h_{L,i}$, at all token positions. It is simultaneously trained to fit an unembedding matrix, $W_U \in \mathbb{R}^{|V| \times d_{\text{model}}}$, which can map any of these final latents $h_{L,i}$ to logit vectors logits_i. Logits are the non-normalized values corresponding to each token, that are collectively fed to the *Softmax* function to turn them into a probability distribution over tokens. In essence, W_U learns a mapping from the transformer's general size for representing one token, d_{model}, to the size of the entire available vocabulary. Importantly, though, W_U was trained to expect that its input latent would contain contextual information pertaining to the entire prior context, already computed (i.e. it was trained to expect activations from the *final layer*, not before). What we want with

[5] For a context of $S = 100$ tokens, that is $100 + 99 + \cdots + 1 = 5050$ Jacobians per layer. $h_{\ell,0}$ contributes one Jacobian per downstream logit, and logits_{99} receives one from each of the 100 latents in layer ℓ that inform it.

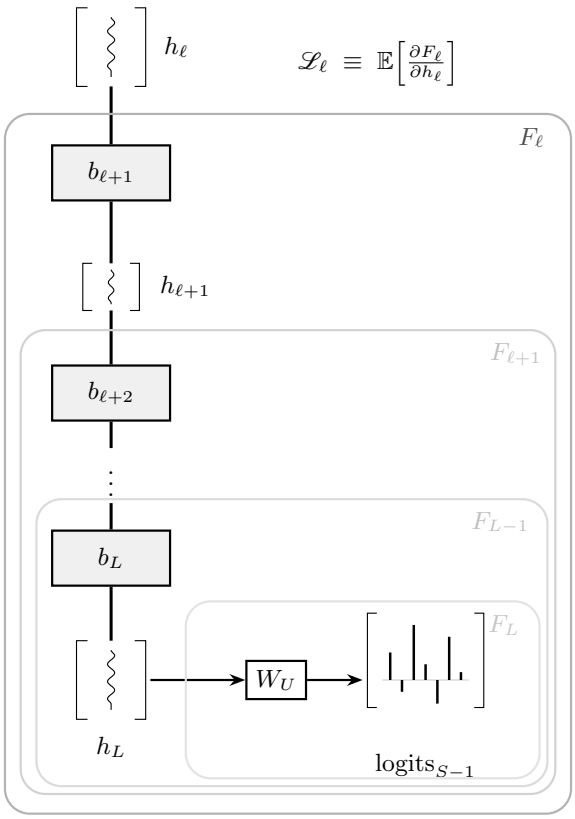

Figure 3: The nested downstream functions: from h_ℓ, everything below is F_ℓ; one layer deeper, $F_{\ell+1} \subset F_\ell$. The lens at layer ℓ is the averaged differential of F_ℓ. Note that F_ℓ includes W_U: the composition lands in logit space, which is why the lens shares W_U's shape, $|V| \times d_{\text{model}}$. The innermost nests make the telescoping explicit: F_{L-1} is b_L followed by W_U, and F_L is nothing but W_U itself—there the differential is exact and context-free; the deeper the tap, the more nonlinearity the lens must average over.

J-lenses is the ability to—in a matrix of the same *shape* as W_U—interpret an "unfinished" latent—that is, a latent at some non-final layer $\ell < L$—for the logits it portends.

Prior research (the *logit lens*) has used W_U to directly decode non-final latents into logits; while at-times insightful, this work ignores the reality of what W_U has been trained to expect as input. The **J-lens** is the principled successor to these prior methods as it faithfully differentiates back through the network, drawing a direct causal link between any latent and the ultimate logit vector it affects.

Why make the **J-lens** differentiate from logits and not from post-*Softmax* probabilities if those probabilities are what are ultimately used to sample the next token? While *Softmax* exaggerates the shape of the raw logit distribution, for the purposes of the **J-lens** and **J-space**—meant to quantify relative directions of disposition in the model's latents—logits will suffice without the additional derivative computation through *Softmax*.[6] A positive causal relationship with some logit entry monotonically translates to a positive causal relationship with the corresponding token's probability of being sampled (all else being equal).

2.4 The Residual Stream

One of the key features of the transformer is its employment of *residual* blocks, as opposed to traditional feedforward layers. A residual layer in a neural network is a function of the form $f(x) = x + g(x)$—where g is the layer's actual transformation (in a transformer, a normalization followed by attention or an MLP)—rather than $f(x) = g(x)$ outright. In essence, residual layers' weights are trained not to fully transform their inputs, but instead to compute *what should be added to* their inputs to achieve some ideal output. One intuition for the plausibility of the construction is that instead of burdening the weights with the task of (a) doing some insightful computation on x and (b) carrying forward the information in x that should be kept, weights only need to learn (a). Another vitally useful feature of residual layers is their role as natural highways for the backward flow of gradients. Following similar logic as the forward-pass intuition, the residual construction provides a kind of ballast for backward-propagating derivatives to be able to reach ever earlier layers without exploding or vanishing through weight matrices. Differentiating the residual form makes this plain: $\frac{\partial f}{\partial x} = I + \frac{\partial g}{\partial x}$. Every residual layer's Jacobian is the identity plus a learned correction, so the Jacobian of an entire stack of layers is a product of near-identity factors—an unbroken causal highway from any latent to any later one. This fact will bear directly on the **J-lens**.

Residual layers have been utilized before the transformer successfully, but the transformer's entire architecture embraces the construction as first class. The **Residual Stream** is the conventional name attributed to the *additive* paths

[6] Which, in fact, warps the confidence of the distribution, skewing the **J-lens** based on exponential confidence.

that course the computational graph of the transformer network. Transformer blocks/layers are typically referred to as "reading from" and "writing to" the residual stream, connoting the stream itself as a kind of progressive *state* of the network. Of course, you could interpret values passing from layer-to-layer of any non-residual network as representations of state, but when the entire network is emphatically trained on the residual paradigm, these residual vectors can more faithfully be understood as more than mere "intermediate" representations.

In keeping with the conventions mentioned above, **J-lens**es tie causal links from logits back to *residual* activations—i.e. vectors at different spots in the residual stream. They, more than intermediate vectors *within* blocks, naturally represent states in the network's forward pass. The **J-lens** then gives us interventional and observational insights into *what the model has the propensity to do* given its progressive states.

3 J-lens

The **J-lens** $\mathscr{L}$ is the key object per layer that allows all the analysis that follows. Taking the *shape* of W_U ($\in \mathbb{R}^{|V| \times d_{\text{model}}}$), it faithfully maps latents $h_{\ell,t}$ to downstream $\text{logits}_{t'}$ (for $t' \geq t$) in the form of a Jacobian.[7] Strictly, $\mathscr{L}_\ell \equiv \mathbb{E}\left[\frac{\partial F_\ell}{\partial h_\ell}\right]$, telling us, in each row v ($\in \mathbb{R}^{d_{\text{model}}}$), the gradient of $\text{logits}[v]$ with respect to h_ℓ. These rows, in essence, represent the directions we'd want to push h_ℓ in order to make $\text{logits}[v]$ higher (i.e. to make token v more likely to be verbalized downstream). If we take activations h_ℓ to be *arrows* as opposed to *points*, we can then measure the *alignment* between the current direction of h_ℓ and that of steepest logit-increase for any token we please. The product $\mathscr{A}_\ell = \mathscr{L}_\ell \cdot h_\ell$ gives us, at each vocabulary index v, the *alignment* between the direction of h_ℓ and the direction in which to move to increase $\text{logits}[v]$. $\mathscr{A}_\ell$, then, acts intuitively like a general compass in token space from the perspective of h_ℓ. Figure 4 pumps intuition about the relationships $\mathscr{L}$ spells out and how they can be used, unpacked further in §**??**.

As the computation below will detail, any $\mathscr{L}_\ell$ where $\ell < L$ is a local linear approximation of a relationship that is by no means linear. Further, any $\mathscr{L}_\ell$ whatsoever is an *average* local linear approximation of the relationship in question. The set of Jacobians averaged to produce a single $\mathscr{L}_\ell$ are all themselves entirely accurate, but only applicable to the specific h_ℓ that allowed its computation. F_ℓ is (for $\ell < L$) by-design nonlinear, so the transformation it performs on h_ℓ—the transformation we'd like to represent with a single Jacobian matrix $\frac{\partial F_\ell}{\partial h_\ell}$—is *dependent on* h_ℓ and irreducible over h_ℓ-space without averaging. Now the averaging, for all the specific accuracy it sheds, makes $\mathscr{L}_\ell$ flexible over token positions and—when further averaged over input sequences **C**—flexibly relevant across semantic contexts.

[7] Because the averaged lens is bound to no position, we suppress position indices from here—writing simply h_ℓ and logits—and reinstate t, t' only where the averaging itself is under discussion, in §3.1. Indexing convention: subscripts on logits denote token positions, brackets select a vocabulary entry—$\text{logits}_{t'}[v]$ is entry v of position t''s logit vector.

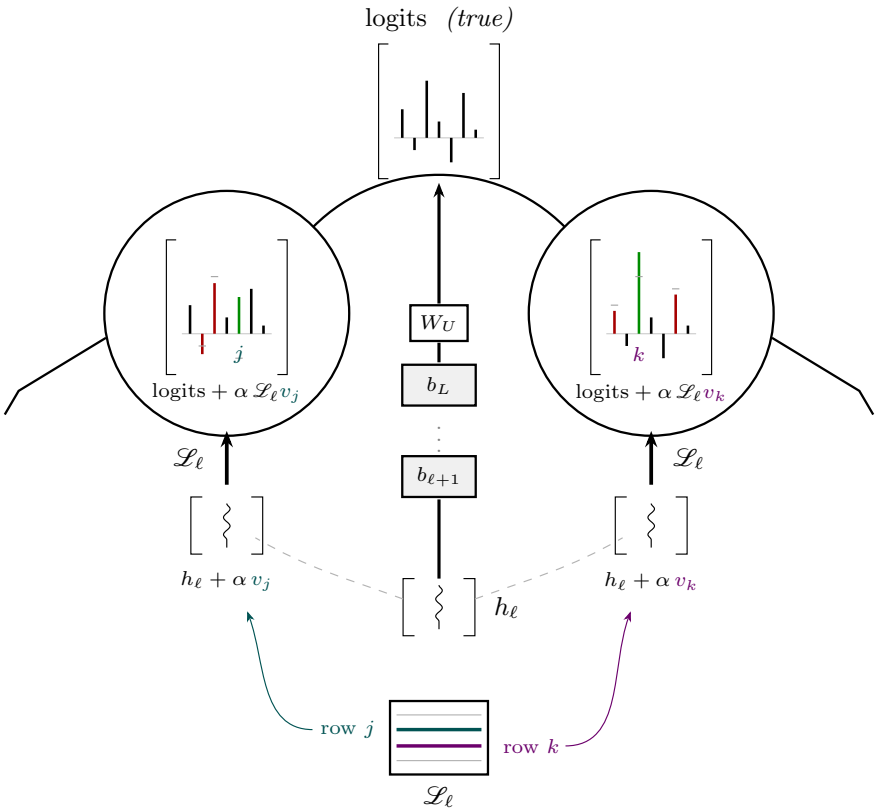

Figure 4: Counterfactual vision, oriented as vision: the latents stand in front of the glasses, and *in* each glass you see logits. The true state h_ℓ (center) earns its logits the hard way—up through every remaining block and W_U. The flanking states are tweaked copies of h_ℓ, each nudged along a row of the lens matrix $\mathscr{L}_\ell$ itself (rows j and k, drawn out of the matrix at bottom)—and they never run the network at all: through the glass, $\Delta\text{logits} = \mathscr{L}_\ell\,\Delta h$, one matrix multiply. Green bars were raised by the tweak, red lowered; gray ticks mark the true heights. Each eye's tall green bar sits exactly at the token whose row it was steered with.

Finally, as §4 will show, the **J-space** decomposition $\mathscr{S}_\ell$ of a latent h_ℓ is constructed with the rows of $\mathscr{L}_\ell$. Thus, $\mathscr{L}_\ell$ is the prerequisite object for **J-vision** broadly.

3.1 Computation

To compute $\mathscr{L}_\ell$ for layer ℓ, we must start with a single Jacobian $J_\ell = \frac{\partial \text{logits}}{\partial h_\ell}$ ($\in \mathbb{R}^{|V| \times d_{\text{model}}}$), taken at one specific latent h_ℓ. J_ℓ is built iteratively, looping over one of two ranges: $|V|$ or d_{model}, whichever is smaller. In the former case, we seed $|V|$ backward walks with each standard basis vector e_v of $\mathbb{R}^{|V|}$ to get each v-th row of J_ℓ. In the latter, we seed d_{model} forward passes (where each layer's 'weights' are local Jacobians, frozen at the cached activations of $F_\ell(h_\ell)$) with each standard basis vector e_d of $\mathbb{R}^{d_{\text{model}}}$ to get each d-th column of J_ℓ. The Jacobian we want to build is the full $\mathbb{R}^{|V| \times d_{\text{model}}}$ matrix, representing what happens to h and which logits vector it produces. In order to unambiguously represent the $\mathbb{R}^{d_{\text{model}}} \to \mathbb{R}^{|V|}$ responsible for that mapping, we have to probe it dimension-by-dimension. These standard basis probes are tracer dyes through the opaque local transformation, one dye per channel. Figure 5 depicts this computation exactly.

As we've said, the ultimate $\mathscr{L}_\ell$ matrix is an average of these J_ℓ matrices over token positions and contexts. Assuming a dataset of $N = 1000$ contexts $\mathbf{C}_n$, each with sequence length $S_n = 100$ tokens, we can conceptualize $J_{\ell,\mathbf{C}_n}$, an average of $(100 + 99 + ... + 1 =)$ 5050 individual J_ℓ matrices, one for each ($h_{\ell,t}$, $\text{logits}_{t'}$) pair in $\mathbf{C}_n$. As mentioned above, we average over token positions so that $J_{\ell,\mathbf{C}_n}$ represents the relationship between $h_{\ell,t}$ and all future ($t \leq t' < S$) logit vectors in the context $\text{logits}_{t'}$. This gives it bearing over future—not past, and not just *immediate* future—verbalization.[8]

[8]While this section depicts mathematically what must be computed and how, in practice, there are several optimizations that sacrifice no mathematical precision. For a single $\mathbf{C}_n$, we do not need $5050 \times |V|$ nor $5050 \times d_{\text{model}}$ passes—only $|V|$ or d_{model}, backward or forward, respectively. Two facts make this true.

1. Taking the gradient of $\text{logits}_{t'}[v]$ back to layer ℓ already natively populates gradient contributions for all $h_{\ell,t}$ ($t \leq t'$). These gradients accumulated in each $h_{\ell,t}$ are naturally inversely proportional to $\frac{t}{S}$ in weight, as earlier latents have more downstream verbalization leverage. One could normalize against this if desired; neither are strictly "correct". The position axis for source latents h_ℓ is thus free—this, alone, reduces 5050 to 100 (under our stated $S = 100$).

2. We can stack one-hot probes at all S token indices at once, in one stapled matrix, liberating the position axis for target $\text{logits}[v]$ as well. This means 1 pass for each token index v or model dimension d fills all latents h_ℓ with their Jacobian contribution for the v or d in question.

An entire context, $\mathbf{C}_n$, then needs either $|V|$ backward or d_{model} forward probes. In the latter case (common in real LLMs), further optimization is available. The nonlinearity we're trying to wrangle in to J_ℓ comes from the layers preceding W_U, as W_U is just a clean linear matrix. Let R_ℓ represent the nonlinear map from h_ℓ to h_L—i.e. F_ℓ without W_U. Then $F_\ell = W_U \circ R_\ell$. To get the full $J_\ell = \frac{\partial \text{logits}}{\partial h_\ell}$, we can just compute $\frac{\partial R_\ell}{\partial h_\ell}$ ($\in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$) and left-multiply it with W_U (a constant matrix, $\mathbb{R}^{|V| \times d_{\text{model}}}$). We recover the full $J_\ell = W_U \cdot \frac{\partial R_\ell}{\partial h_\ell}$, dealing with the larger dimension, $|V|$, only once as opposed to within each of the d_{model} iterations.

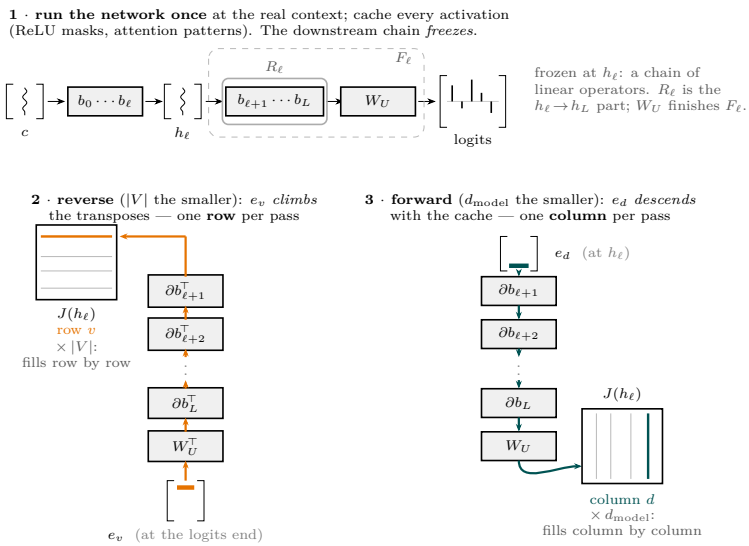

Figure 5: Two ways to read the frozen chain. One forward pass at the real context anchors everything: it fixes each layer's local linear behavior, and from boundary ℓ down, the differentiated network is a chain of linear operators. The cached activations are the necessary shared ingredient, and both modes reuse the one cache for every pass. Reverse mode seeds a one-hot e_v at the logits and *climbs* the chain's transposes back to h_ℓ: one row per pass, $|V|$ passes, J filled row by row. Forward mode releases a tangent one-hot e_d at h_ℓ and *descends* alongside the cache: one column per pass, d_{model} passes, J filled column by column. The one-hots occupy the seed slot in both directions, h_ℓ stays put, anchoring *where* the Jacobian is taken. Two equivalent tracer dyes through each channel of $F_\ell = W_U \circ R_\ell$. For $(|V|, d_{\text{model}}) = (50\text{k}, 12\text{k})$, forward mode is the $4\times$ discount.

Figure 6 and Figure 7, respectively, show the two mirror ways of conceptualizing the computation over the set of latent-logit pairs in a single context. In the former, we look at a single logit vector logits_{S-1} and each latent $h_{\ell,t}$ that feeds it from layer ℓ across all sequence positions $t < S$. In the latter, we see things from the perspective of a single latent $h_{\ell,0}$ and all S logit vectors (and partial derivatives) it affects.

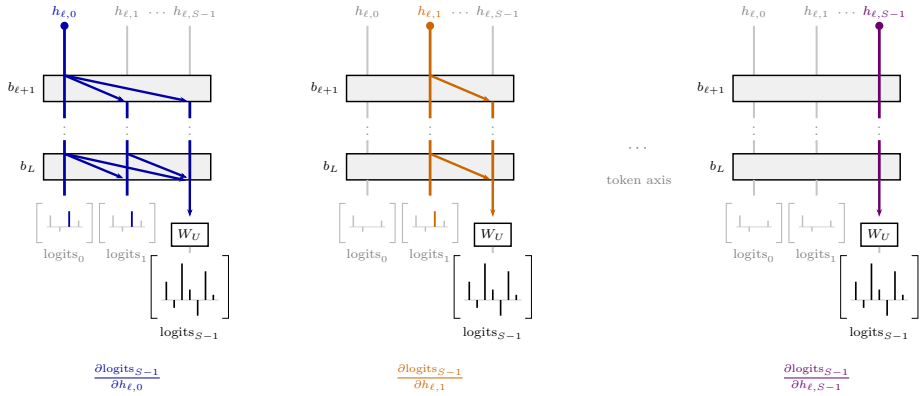

Figure 6: One stack, S streams, three of the taps—read the panels *at once*. Inside every downstream block, a latent is delivered to *every* later position (the diagonals): $h_{\ell,0}$ floods all streams, $h_{\ell,1}$ all but position 0's, $h_{\ell,S-1}$ only its own. The faint charts mark the logits each tapped latent feeds—colored bars where its influence lands, so the reach shrinks left to right. The first latent affects all downstream logits; the last logits owe to all upstream latents. These (source t, target $t' \geq t$) pairs are exactly the Jacobians the **J-lens** averages over both position axes.

$\mathscr{L}_\ell$ is then the average of N mean Jacobians $J_{\ell,\mathbf{C}_n}$ per context. Finally, we compute $L+1$ of these $\mathscr{L}_\ell$ matrices per network (the last of which, $\mathscr{L}_L$, is nothing but W_U itself)!

3.2 Insightful Caveats

What is the epistemological cost of averaging the local Jacobians into a global lens? An individually computed J_ℓ (Jacobian of $F_\ell : \mathbb{R}^{d_{\text{model}}} \to \mathbb{R}^{|V|}$) is exact, but indexed to a specific point in layer ℓ's activation space. The earlier ℓ is, the more of F_ℓ's higher-order curvature is being averaged away when we mean—first over positions into each context's $J_{\ell,\mathbf{C}_n}$ and then over contexts—into the single $\mathscr{L}_\ell$. We average the map over all $(N \times S)$ visited/sampled activations, pretending like local linear behavior is shared regionally when it really isn't. Figure 8 attempts to stimulate your intuitions about the information lost in this local linear averaging.

So then why average over contexts if we're creating an increasingly approximated object by doing so? The goal of $\mathscr{L}$ is not to be an exact J of a specific logits vector, with respect to a specific h—it is manifestly to be an aggregate,

Finally, instead of each $\mathbf{C}_n$ alone needing $|V|$ or d_{model} passes, we can batch contexts together, obtaining our entire $\mathscr{L}_\ell$ in $|V|$ or d_{model} (typically the latter) passes total per layer.

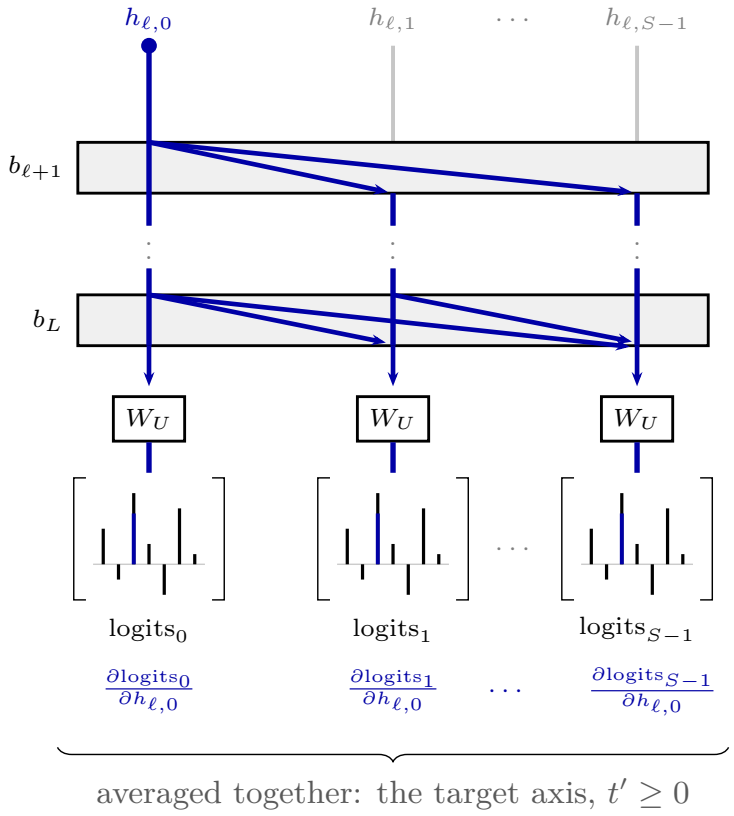

Figure 7: The same pairs, grouped the other way. Figure 6 is oriented toward one logit vector—the causal *heads* feeding logits_{S-1}, one panel per source. Here, the dual orientation: one latent, $h_{\ell,0}$, and every logit vector it touches—its causal *tails*. Every stream admits the same shared W_U, so the tapped latent's target axis is this whole fan, one Jacobian per downstream chart (blue bars mark its influence landing). Group the $(t,\, t' \geq t)$ pairs by target or by source—the set, and the mean the **J-lens** takes over it, is identical.

generalized map. While the complex field of local functions defined in F—the distinct attention paid to every unique context—is exactly what makes these models so powerful, the construction of $\mathscr{L}$ is a *postulation* that there is some shared shape across the ways each context inspires speech in the model. Complex as F necessarily is, though, it is not some incompressibly arbitrary map.[9] We lose information when we average local specificity into regional (dare we say) global approximations, but what we retain has general, nontrivial causal insight (as the paper shows and we discuss in §5.2).[10]

Averaging is a hypothesis. Here is how we test it: we can measure the magnitude of difference ($\delta_\ell = \|J_\ell(h_\ell) - \mathscr{L}_\ell\|^2$) between each local J_ℓ and the global $\mathscr{L}_\ell$, and then average this to get an overall expected **dispersion** measure ($\mathscr{D}_\ell = \mathbb{E}_h[\delta_\ell]$) for a given $\mathscr{L}_\ell$ computed over a sampling of contexts. $\mathscr{D}_\ell$ is conceptually proportional to (1) variance in the distribution of activations h_ℓ and size of the region of F_ℓ's field sampled across contexts, and (2) the higher-order curvature of F_ℓ in the h-region sampled. These factors compound, schematically: $\mathscr{D}_\ell \propto \left\| \frac{\partial^2 F_\ell}{\partial h_\ell^2} \right\|^2 \cdot \mathbb{E}\|h_\ell - \bar{h}_\ell\|^2$.

$\mathscr{D}$ fills in the detail productively smeared by $\mathscr{L}$, complementing but not invalidating its causal importance.[11]

4 J-space

4.1 Computation

Sparse nonnegative decomposition. How (outer and inner loop, gradient descent reappears), why, why nonnegative?

4.2 Insightful Caveats

Decomposed **J-space** vector only covers up to 10% of variance of activations.

5 J-vision In Language Models

The **J-lens** is only day one. We need to expand interpretability, safety, training, and transparent use with **J-vision**.

[9] Don't you feel that, even though every facial expression you've made is the outcome of a unique circumstance, there might still be an average relationship your best friend could use to predict what you may say? Transformers have no faces, they have latent activations; interpretability researchers armed with calculus are the knowing friends who seek to learn their behavior. But your friend doesn't study the physics of your facial expressions to learn how they entail behavior—they build an averaged, general mapping between facial expression, mood, and your behavior.

[10] For what it is worth, most *training* regimens adhere to the same hypothesis with respect to the effect of averaging over nonlinearity. It is hard to deny its success in doing so. $\mathscr{L}$ takes the same kinds of means and shows that—in the trained model, in *activation* space—specificity is not the entire story.

[11] Good, knowing friendships are not based on facial physics!

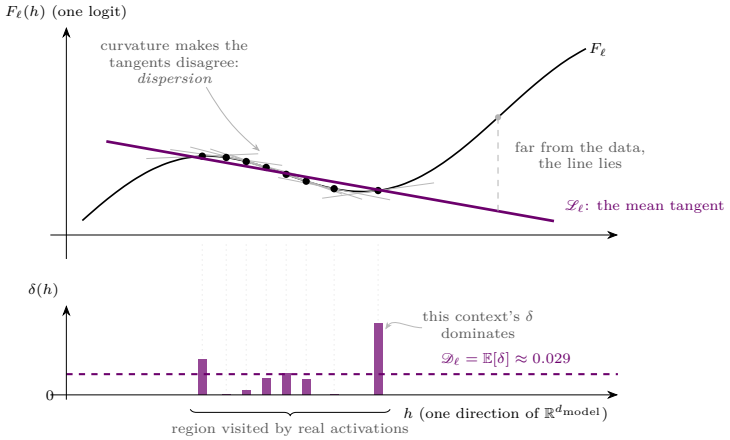

Figure 8: What the lens is: the average of the local linear behaviors $J_\ell(h) = \frac{\partial F_\ell}{\partial h}$ at the activations the model actually visits (dots; one gray tangent each). Where F_ℓ curves within the visited region, the tangents fan—that spread *is* the dispersion $\mathscr{D}_\ell = \mathbb{E}\|J_\ell(h) - \mathscr{L}_\ell\|^2$. And the mean tangent is a local instrument: far from the sampled region, the line and the function part ways. The lower panel makes the spread concrete: each bar is one visited context's $\delta = \|J_\ell(h) - \mathscr{L}_\ell\|^2$—how far its tangent tilts from the mean—and $\mathscr{D}_\ell$ is simply their average, pulled up here by the lone high-curvature context at the region's right edge.

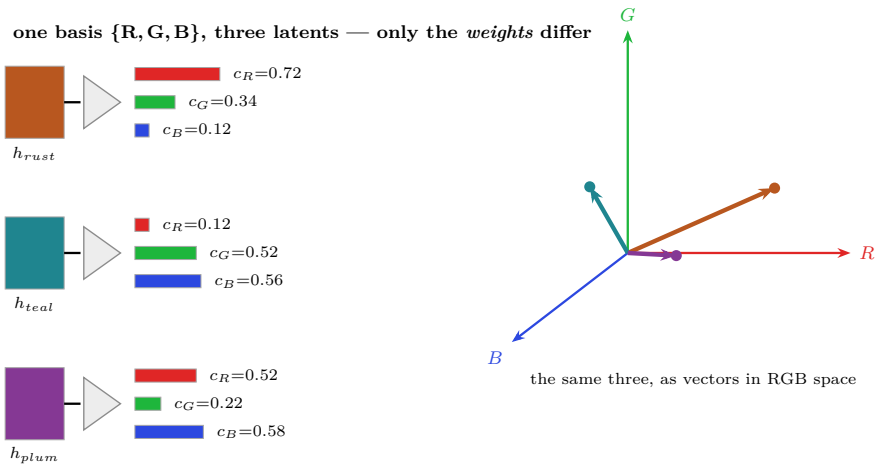

Figure 9: **A basis, to warm up.** A "complex color" latent h splits—through the prism—into fixed primaries R, G, B; the *length* of each bar is that primary's coefficient. Three different latents (rust, teal, plum) use the *same* three directions and reconstruct *exactly*—what distinguishes them is only the weights (c_R, c_G, c_B), read off as the bar lengths, or as the coordinates of each arrow in the RGB cube (right; axes and arrows carry their own color). Three directions, minimal and spanning—a true basis. The J-lens dictionary is neither.

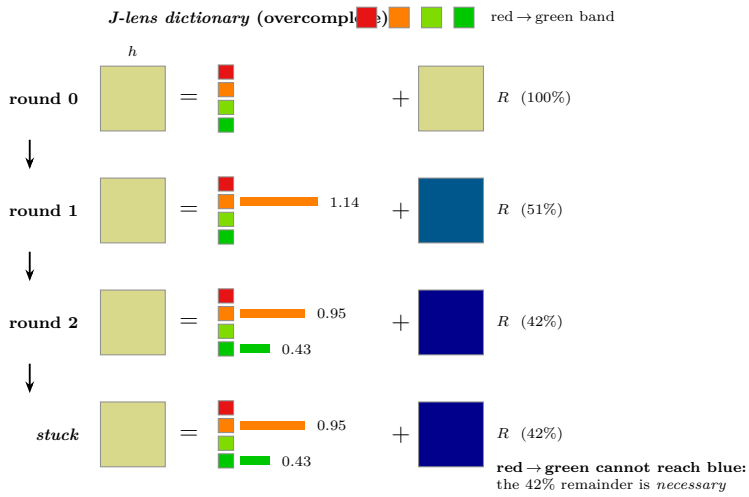

Figure 10: **Not a basis: overcomplete, yet unable to span.** Matching pursuit as successive equations $h =$ (dictionary decomposition) $+ R$, read top to bottom. Each round greedily grabs the band hue best aligned with the current residual R and refits the weights (round 2 pulls orange $1.14 \to 0.95$ as green enters—the inner loop). The residual keeps losing color—but only *to a point*: a red-through-green band cannot represent blue, so R stalls at pure blue and 42% of h is a *necessary* remainder that no further round can touch. The dictionary is redundant—four hues, only two ever used—*yet* it still cannot span. Contrast the true basis of Figure 9: minimal *and* spanning, exact. The J-lens is the opposite on both counts.

5.1 J-action

High leverage, directed intervention!

5.2 Findings

Swaps, nudges, blocks, safety probing.

5.3 Further Questions Askable

Tracing dispositional alignment as abstraction, tracing robustness of dispositional abstraction.

6 J-vision, Generally

As I wrote this paper, I stepped out to pick up dinner and indulged in a whistle-along car ride. I am a good whistler, but tonight there was unusual accord between the sounds I wanted and those that were broadcast from my mouth. It stirred me to think about the control my brain was exerting over many distinct modules of my body. The physics of human whistling are far beyond my competency, and I know for sure its theorems and equations are etched nowhere in my brain. All that my brain needs to know to realize my conviction—which we

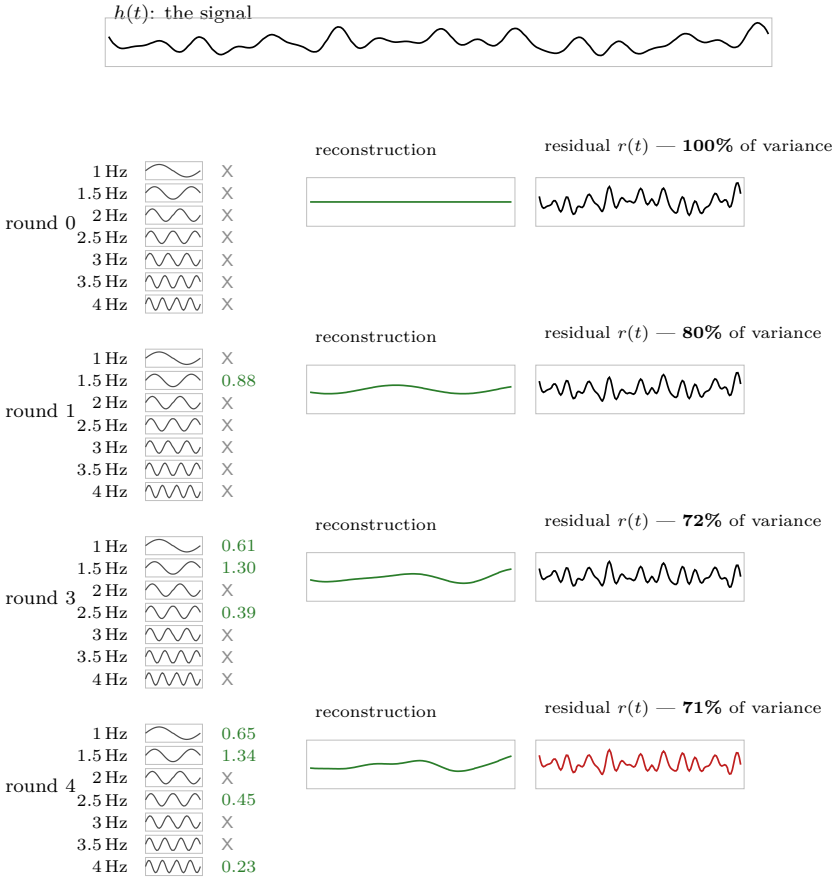

Figure 11: **The same story, faithful: frequencies.** Matching pursuit's native domain. The signal h (top) is decomposed against an *overcomplete* dictionary of pure tones—the 1–4 Hz band sampled every 0.5 Hz, twice as fine as a 1-second window can resolve, so the atoms are redundant and *non-orthogonal*. Each round grabs the best-matching tone and refits *all* active magnitudes by least squares: watch 1.5 Hz climb $0.88 \to 1.30 \to 1.34$ as neighbours enter (the inner loop), and note redundant rows ($2, 3, 3.5$ Hz) left X, never needed. Crucially, h's energy lives mostly *above* the band, which no atom reaches—so the reconstruction (green) stays small and the residual barely shrinks: after four rounds it still holds 71% of the variance, a *necessary* remainder. Overcomplete, redistributing, and capturing only a sliver—the honest mirror of the J-lens (contrast the exact basis of Figure 9).

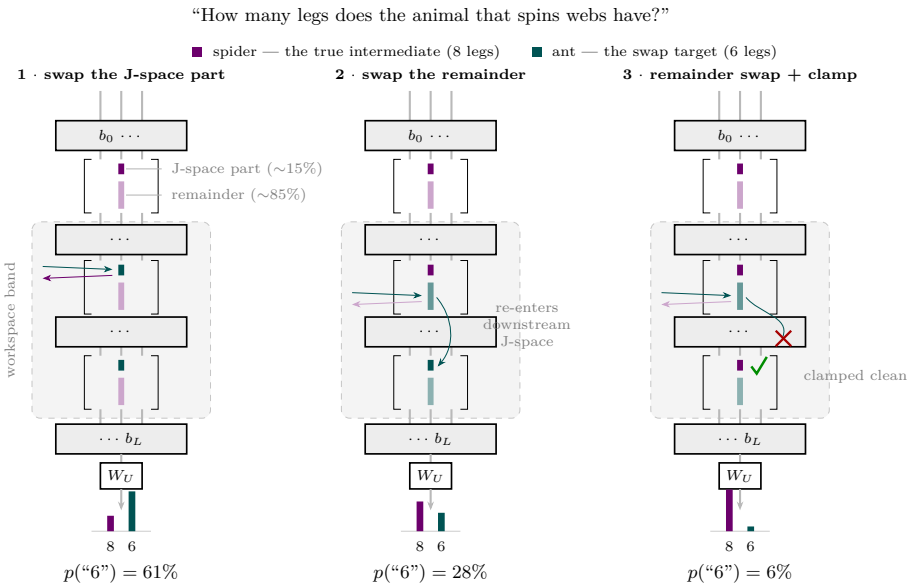

Figure 12: The three-pronged swap. The probe direction for the unspoken intermediate (spider) splits into a J-space part—a sparse nonnegative combination of k=25 lens rows, only ∼15% of the probe's variance—and the ∼85% remainder. All interventions act at every layer of the workspace band, at every token position. **(1)** Swapping just the J-space part for ant's flips the answer 61% of the time. **(2)** Swapping only the remainder still works 28% of the time—but only because downstream layers read the moved content and re-express it into their own J-space (teal arrow). **(3)** Same remainder swap with the downstream J-space coordinates clamped to their clean (spider) values: the re-entry is blocked and the effect collapses to 6%. The ∼15% carries the causal freight; the ∼85% moves behavior only by being re-admitted into J-space.

won't bother interrogating today—is a **J-lens** differentiating *whistled frequency* with respect to what I am loosely imagining as a stream of *electrical command vectors* propagating from my brain to my diaphragm, lungs, throat, lips, and tongue. The algorithm does not need realization, only $\mathbb{E}[\frac{\partial \textbf{Pitch}}{\partial \textbf{Command}}]$ does (and even this quantity is not *realized*, but baked into the machinery).

Why $\mathbb{E}$? I can whistle with similar aptitude across many different physical contexts; wet lips after a shower, dry lips on a cold winter day, laying on my back, walking on the treadmill, engaging my tongue or not, optimizing for volume or exactness...In each context, the machine to control is slightly different, but the function that describes how to change notes in any of these contexts is overwhelmingly quite similar. Whatever the realization of the whistling **J-lens** in my brain, it is an average over contexts. I can pick up whistling without having any conscious awareness of the moisture on my lips or in the air. Sure, once this information is known, there's some steering and tuning that occurs (the **J-lens** is not all-seeing, the **J-space** does not exist or act alone!), but there is a directional map of where to go in *electrical command space* to reach a desired location in *whistled frequency space*.

One more feature of the whistling lens bears mentioning here, as it relates to robustness of disposition. The map from *command space* to *frequency space* is *non-injective,* meaning it exhibits **many-to-one** relationships. There are many, many specific commands my brain can stream that will achieve the same frequency, especially when the diverse contexts above are considered. There is, then, a notion of *fibers* with respect to this map. There are directions in command space that can be traversed that will not change the corresponding output in frequency space. If I want to stay on tune and remain alive, I must breathe through my performance; my brain evidently has found invariant fibers along which it should travel in command space as the periods of respiration occur. The same idea goes for the moisture of my lips, which is constantly changing, especially when I whistle. There are circuits in the brain that can pin the required invariant—pitch—with a proficient honoring of the fibers admitted under the **J-lens**. **J-vision** gives us a new intuitive vocabulary with which to explain cybernetics—in differentiable and non-differentiable machines alike.

The mystery of how high-level mental goals filter down through mechanism into realization has always tantalized me. **J-vision** is the beginning of the answer. Anthropic's results suggested that Claude's downstream weights specifically amplify privileged **J-space** directions internally, and we have to imagine that such a privileged space (in any layer ℓ) developed in concert with other parameters (in layers $\ell' < \ell$) that learned to exert high-leverage influence on the future behavior of the model by manifesting changes in h_ℓ. But beyond a system's internal apprehension (by way of **J-vision**) of its high leverage states, we as machine learning engineers are tantalized by *our own* interventional opportunities as *external* agents. If the brain has modules akin to language models, we can imagine that higher-level systems have found their own levered hooks informed by **J-vision** as well that can enable executive steering.

J-vision is both the generalized ability to map changes in high-leverage control variables to those in downstream domains of behavior *as well as* the

disposition to think about and look at hierarchical systems in this framing. **J-vision** seeks **J-vision**. We've just gotten our first **J-glimpse**.